

Universidad Nacional Autónoma de México



Facultad de Ingeniería

División de Ingeniería Eléctrica



Laboratorio Computación Gráfica e Interacción
Humano-Computadora (6590)

Práctica 8: Iluminación 2

Nombre: Calderón Gutiérrez Victor Emiliano

No. Cuenta: 423097331

Grupo (laboratorio): 03

Grupo (teoría): 02

Semestre 2026-1

Fecha de entrega: 26/10/2025

Calificación: _____

1. Ejercicios

En la realización de esta práctica se hicieron uso de muchos de los conceptos e incluso trabajo hecho en prácticas anteriores. Se realizaron 3 ejercicios los cuales son todos relacionados con luces, pero ahora no solo con una luz, más bien con sistemas más complejos.

1.1. Ejercicio 1

El primer ejercicio implementó cosas que vimos en prácticas al inicio del semestre. Observamos como trabajar con el modelo jerárquico para de alguna forma llevar la posición del objeto que tenemos y conforma este se vaya actualizando entonces podemos llevar luces, este es para el cofre, para lo cual se tuvo que crear una luz que se va moviendo con esta posición relativa del cofre y además se liga con la posición en la que se encuentra, si está cerrado o abierto. Se uso un color morado para crear esta luz.

Código

Primero se declaró la luz:

```
unsigned int spotLightCount = 0;
// Luz 0: Luz del Cofre (morada)
spotLightsF[0] = SpotLight(0.7f, 0.1f, 0.9f,
    0.0f, 4.0f,
    0.0f, 0.0f, 0.0f,
    0.0f, 1.0f, 0.0f,
    0.5f, 0.1f, 0.0f,
    25.0f);
spotLightCount++;
```

Luego se hizo la lógica para atrapar la luz y pegarla al cofre del automóvil. Se creó una variable para llevar las luces del coche (se usa más a detalle en el ejercicio 2).

```
// Variable base para la posición de todas las luces del coche
glm::vec3 posCoche = glm::vec3(0.0f + mainWindow.getmuevex(), 2.0f, -4.0f);

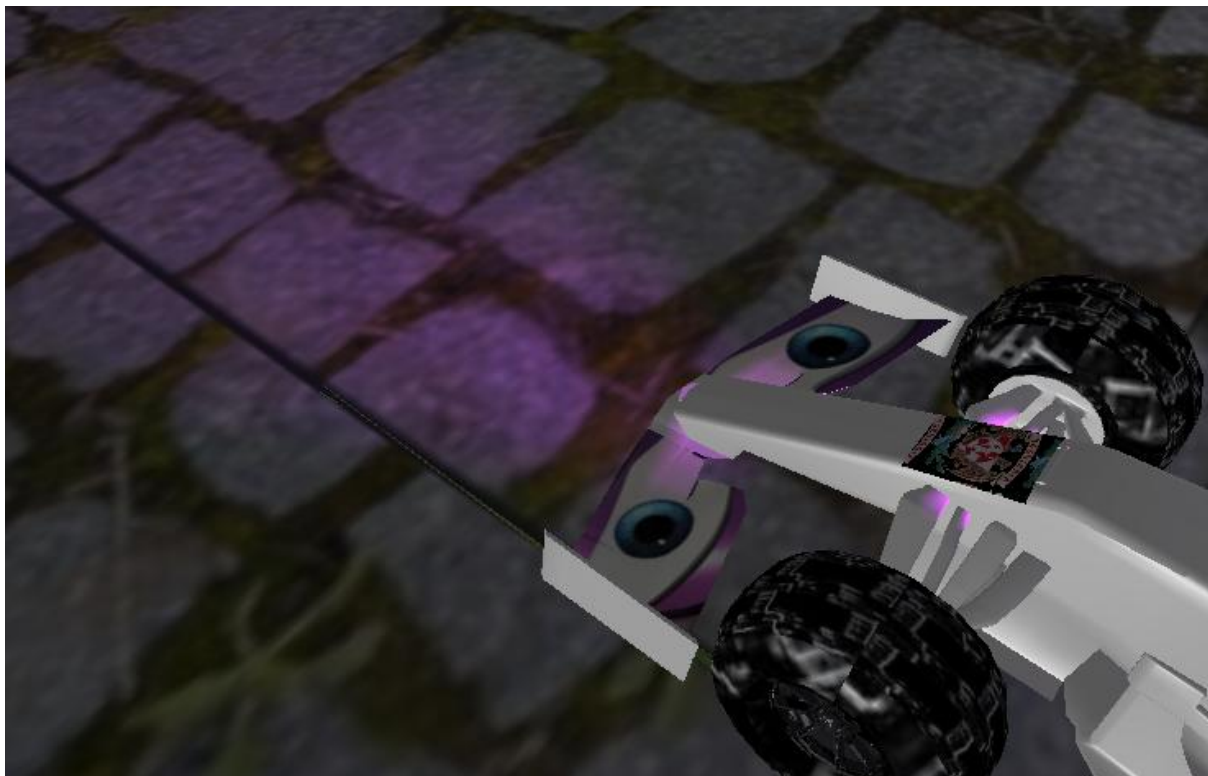
// Ejercicio 1: Actualizar luz del Cofre (Morada)
glm::vec3 posLuzCofre = posCoche + glm::vec3(0.0f, 0.0f, 0.0f);
float anguloCofre = mainWindow.getAbrirCofre() - mainWindow.getCerrarCofre();
glm::mat4 rotMatrixCofre = glm::rotate(glm::mat4(1.0f), glm::radians(anguloCofre), glm::vec3(0.0f, 0.0f, -1.0f));
glm::vec3 dirLuzCofre = glm::vec3(rotMatrixCofre * glm::vec4(-1.0f, 0.0f, 0.0f, 1.0f));
spotLightsF[0].SetFlash(posLuzCofre, glm::normalize(dirLuzCofre));
spotLightsT[0].SetFlash(posLuzCofre, glm::normalize(dirLuzCofre));
```

Para abrir y cerrar el cofre se hizo uso de las variables creada en la práctica en donde se trabajó con el cofre, solo se modificó el uso de las teclas, ahora se definió con Z y X.

```
}  
if (key == GLFW_KEY_Z)  
{  
    theWindow->cofre_abre += 10.0;  
}  
if (key == GLFW_KEY_X)  
{  
    theWindow->cofre_cierra += 10.0;  
}  
if (key == GLFW_KEY_ESCAPE)  
{  
    theWindow->cofre_abre = 0.0;  
    theWindow->cofre_cierra = 0.0;  
}
```

Capturas

Al correr el código se observa la luz morada que se queda pegada al cofre al presionar las teclas.





1.2. Ejercicio 2

Para el segundo ejercicio, se hizo un trabajo a mi parecer mucho más interesante, ya que este en particular requería tener dos arreglos diferentes, al tener estos dos arreglos se permitió tener la capacidad de prender tanto el primero y el segundo por separado, aquí solo se implementaron dos diferentes y de acuerdo con la entrada se determinaba cual de los dos prendía.

Código

Primero se crearon los spotlights que se iban a usar, en este caso se hizo uso de los arreglos diferentes, por lo que se crearon dos arreglos, los cuales, en el primer valor guardan la luz

morada del ejercicio anterior y como segundo valor ponen la luz delantera o la trasera, dependiendo el arreglo usado.

```
spotLightsT[0] = spotLightsF[0];

// Luz 1: Luz delantera (verde árbol)
spotLightsF[1] = SpotLight(0.1f, 0.6f, 0.1f,
    0.0f, 8.0f,
    0.0f, 0.0f, 0.0f,
    0.0f, 0.0f, -1.0f,
    0.5f, 0.1f, 0.0f,
    25.0f);
spotLightCount++;

// Luz 1: Luz trasera (ambar)
spotLightsT[1] = SpotLight(0.9f, 0.7f, 0.1f,
    0.0f, 4.0f,
    0.0f, 0.0f, 0.0f,
    0.0f, 0.0f, 1.0f,
    0.5f, 0.1f, 0.0f,
    25.0f);
```

Al hacerlo así, se puede siempre asegurar de tener la luz morada prendida y solo intercambiar el arreglo a usar si se va hacia adelante o hacia atrás. Ahora, se tuvo que implementar la lógica para que al avanzar con C y al retroceder con V pudiera prender o apagar el arreglo a usar.

```
// Ejercicio 2: Actualizar luces Delantera
glm::vec3 posFaroDelantero = posCoche + glm::vec3(0.0f, -0.5f, -1.0f);
glm::vec3 posFaroTrasero = posCoche + glm::vec3(7.0f, -0.5f, 1.0f);
spotLightsF[1].SetFlash(posFaroDelantero, glm::vec3(-1.0f, 0.0f, 0.0f));
spotLightsT[1].SetFlash(posFaroTrasero, glm::vec3(1.0f, 0.0f, 0.0f));
```

Aquí es donde se usó la jerarquía para llevar la posición del coche y así todas las luces están ligadas al lugar del espacio en donde está localizado el coche.

Para prender o apagar las luces se generó un if/else, en donde se definía cuando prendía el primer arreglo y cuando el segundo, cada que se hacía eso se pasaba ese arreglo específico al shader.

```

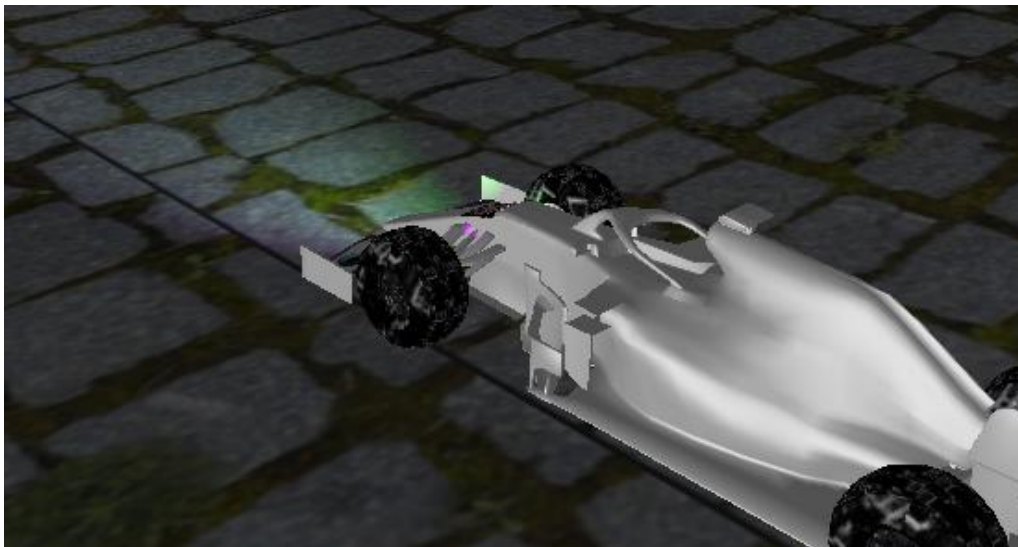
tipoSpotlight = mainWindow.getlucespuntuales();
if (tipoSpotlight < 0.5f)
{
    shaderList[0].SetSpotLights(spotLightsF, spotLightCount);
}
else if (tipoSpotlight < 1.5f)
{
    shaderList[0].SetSpotLights(spotLightsT, spotLightCount);
}

```

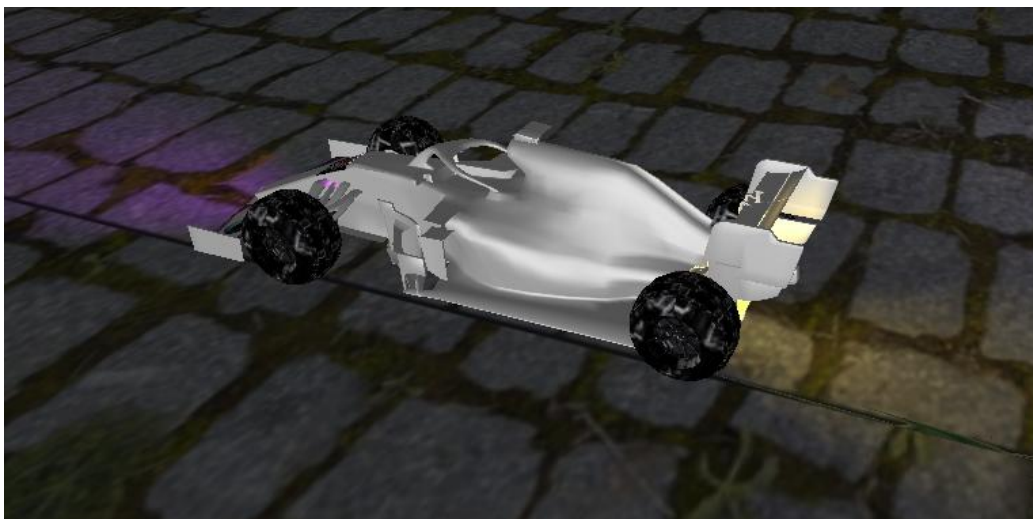
Capturas

A continuación, se presentan algunas capturas de como se ve en acción.

Al avanzar se usó el color verde.



Al retroceder se desactiva ese color y prender un color ámbar en la parte trasera.



Y como se observa, la luz del cofre sigue activa.

1.3. Ejercicio 3

Ahora, para el tercer ejercicio se pidió trabajar con la lampara de la práctica anterior, la cual se iluminó también, para esto adicionalmente se pidió agregar otro objeto que pudiera iluminar, en mi caso use el bastón del vacío, que es un objeto de League of Legends, un objeto que pueden comprar los magos del juego. En este caso fue un poco más complejo trabajarlo, porque intenté aplicar la misma lógica que en el ejercicio anterior, sin embargo, no supe como renderizar los dos al mismo tiempo, por lo que cree un tercer arreglo el cual tenía las dos luces prendidas, así es como se logró trabajar con esas luces.

Código

Para este ejercicio se reutilizó mucho código de otras prácticas, lo complicado fue la lógica para usar los arreglos. Para comenzar se hizo la definición de los arreglos a usar. Aquí es donde se decidió hacer uso de los tres arreglos, para de esta forma únicamente renderizar las luces que se ocupaban para cada uno de los casos.

```
//contador de luces puntuales
unsigned int pointLightCount = 0;
// Lampara
pointLightL[0] = PointLight(1.0f, 1.0f, 1.0f,
    0.1f, 2.5f,
    5.0f, 10.0f, -20.0f,
    0.3f, 0.2f, 0.1f);
pointLightCount++;

// Bastón del vacío (League of Legends)
pointLightB[0] = PointLight(0.4f, 0.0f, 0.67f,
    0.1f, 5.0f,
    20.0f, 6.5f, -20.0f,
    0.3f, 0.2f, 0.1f);

// Ambos pointlights prendidos
pointLightsCombinadas[0] = pointLightL[0];
pointLightsCombinadas[1] = pointLightB[0];
```

Por lo tanto, para prender y apagar las luces se decidió usar lógica como la de actividades anteriores a estas, por lo que se tuvo que poner nuevas variables tanto en Window.cpp y Window.h.

```
GLboolean getprendida() { return luzprendida; }
GLboolean getBastonPrendido() { return bastonPrendido; }
GLboolean getLucesPuntuales() { return lucesPuntuales; }
```

```
}
if (key == GLFW_KEY_O && action == GLFW_PRESS)
{
    theWindow->bastonPrendido = !theWindow->bastonPrendido;
}
```

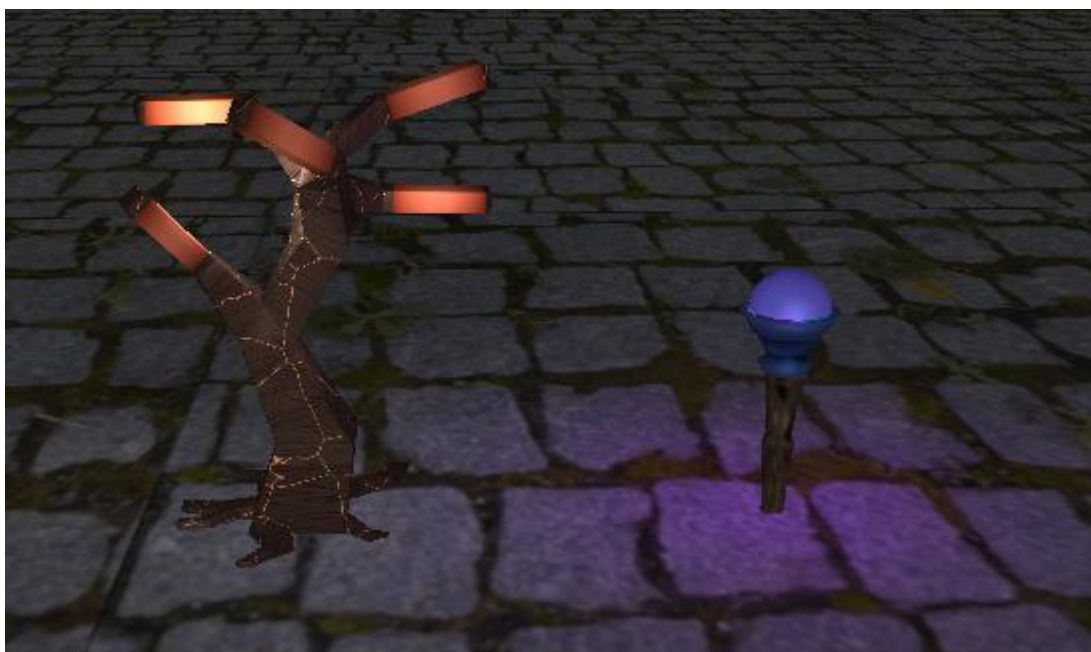
Ya teniendo eso, se pudo implementar un bloque if/else más completo para tener los 4 posibles casos, la lampara prendida, el bastón prendido, ambos prendidos y ninguno prendido.

```
// Ejercicio 3: Prender y apagar luces
if (mainWindow.getprendida() && mainWindow.getBastonPrendido())
{
    // Ambas luces prendidas
    shaderList[0].SetPointLights(pointLightsCombinadas, 2);
}
else if (mainWindow.getprendida())
{
    // Solo la lampara prendida
    shaderList[0].SetPointLights(pointLightL, 1);
}
else if (mainWindow.getBastonPrendido())
{
    // Solo el bastón prendido
    shaderList[0].SetPointLights(pointLightB, 1);
}
else
{
    // Ambas luces apagadas
    shaderList[0].SetPointLights(pointLightL, 0);
}
```

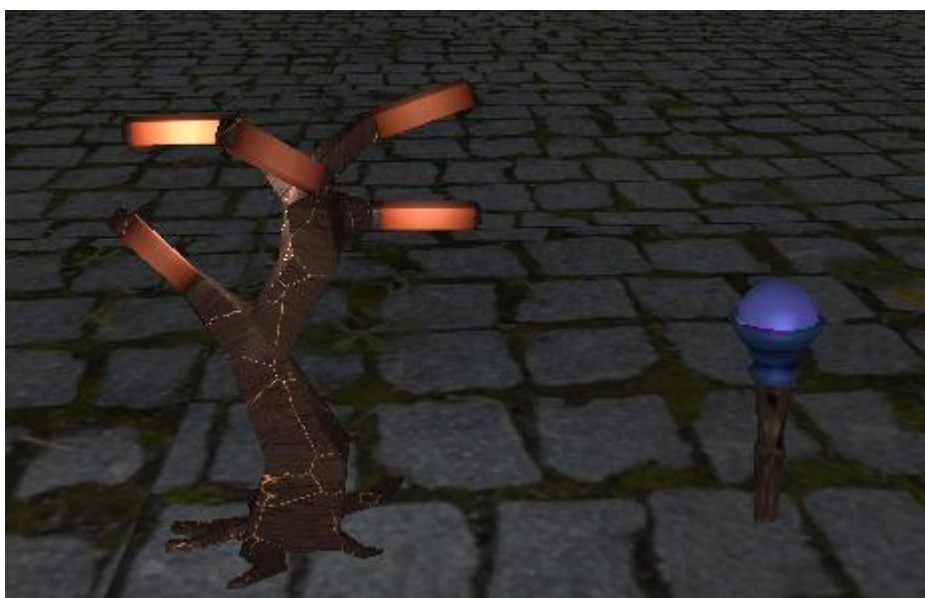
Capturas

Ya que se hizo la lógica se puede probar y ver los cuatro casos en acción.

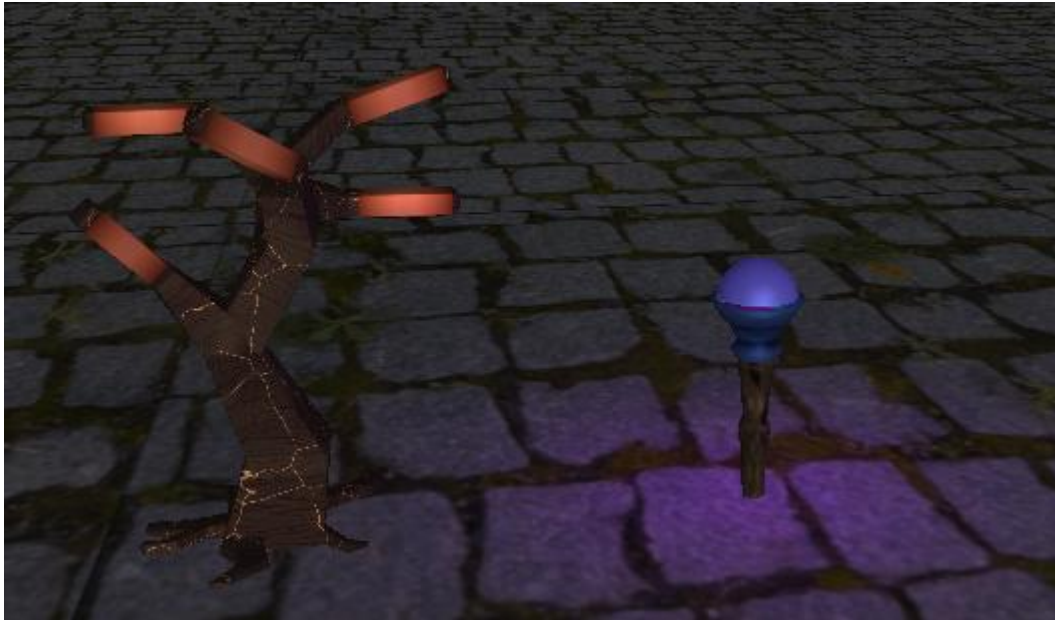
Los dos prendidos:



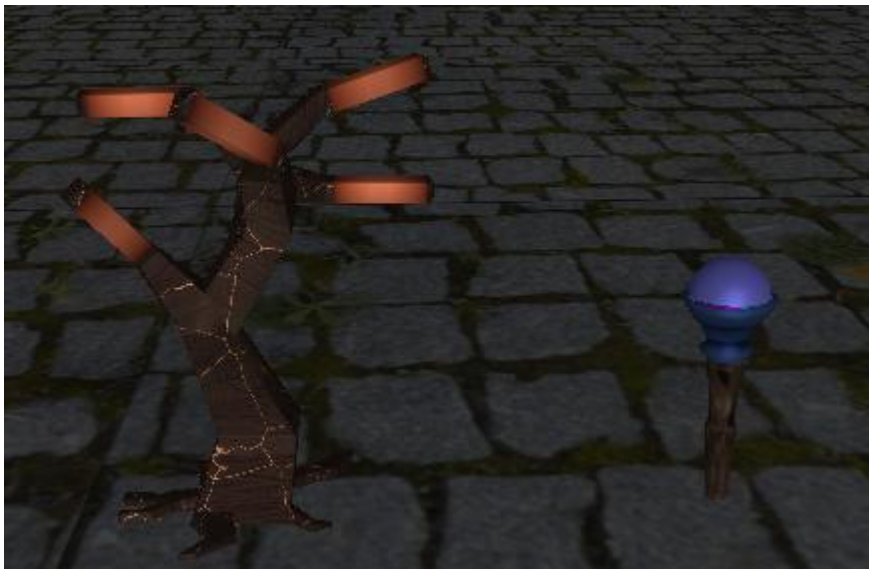
Solo la lampara prendida:



Solo el bastón prendido:



Ambos apagados:



2.Problemas

Esta práctica se me hizo muy completa y los problemas con los que me enfrenté también fueron más complejos. Como se ha visto en las últimas actividades, si existiera algo que no se haya aprendido o entendido en actividades anteriores entonces atrasa el poder realizar las modificaciones. En mi caso, yo tenía un modelo de cofre en donde lo ponía en la parte trasera tal como la fórmula 1 reales, se puede revisar en prácticas anteriores, pero para esta se tuvo que trabajar con un cofre en la parte frontal para poder ponerle una luz. Esto más que ser un problema me ayudó mejor a trabajar con los modelos, las texturas y el exportado e importado de modelos, por lo que me sirvió de práctica para comenzar con el proyecto final.

3. Conclusión

Estas prácticas cada vez se hacen mucho más complejas y completas. Se observa como cada vez se comienza a trabajar con información mucho más compleja y que a su vez no se nota tanto. Igual se puede observar como esto nos ayuda de mucha práctica para mejorar nuestra lógica y entender cual sería la mejor forma de trabajar y buscar siempre optimizar recursos y así evitar que se cargue información innecesaria. En general, las prácticas cada vez se hacen más interesantes y demuestran correctamente la forma en que todo se puede ir usando para generar escenas 3d mucho mejor preparadas.

4. Bibliografía

- Ing. Jose Roque RG. (2025d, octubre 20). *práctica 8 Iluminación grupo 3* [Vídeo]. YouTube. <https://www.youtube.com/watch?v=XyJdJV6hCNk>
- Sketchfab. (s. f.-f). *Void Staff (League of Legends) - Download free 3D model by Amenya (@amenyabm)*. Recuperado 26 de octubre de 2025, de <https://sketchfab.com/3d-models/void-staff-league-of-legends-cedca5b7338548ec8851fe2ffa48a0af>
- *Void Staff | League of Legends Wiki*. (s. f.). League Of Legends Wiki. Recuperado 26 de octubre de 2025, de https://wiki.leagueoflegends.com/en-us/Void_Staff